

Defeating the Training-Inference Mismatch via FP16

Penghui Qi^{*†1,2}, Zichen Liu^{*1,2}, Xiangxin Zhou^{*1},
Tianyu Pang¹, Chao Du¹, Wee Sun Lee², Min Lin¹

¹Sea AI Lab ²National University of Singapore

🔗 <https://github.com/sail-sg/Precision-RL>

Abstract

Reinforcement learning (RL) fine-tuning of large language models (LLMs) often suffers from instability due to the numerical mismatch between the training and inference policies. While prior work has attempted to mitigate this issue through algorithmic corrections or engineering alignments, we show that its root cause lies in the floating point precision itself. The widely adopted BF16, despite its large dynamic range, introduces large rounding errors that breaks the consistency between training and inference. In this work, we demonstrate that simply reverting to **FP16** effectively eliminates this mismatch. The change is simple, fully supported by modern frameworks with only a few lines of code change, and requires no modification to the model architecture or learning algorithm. Our results suggest that using FP16 uniformly yields more stable optimization, faster convergence, and stronger performance across diverse tasks, algorithms and frameworks. We hope these findings motivate a broader reconsideration of precision trade-offs in RL fine-tuning.

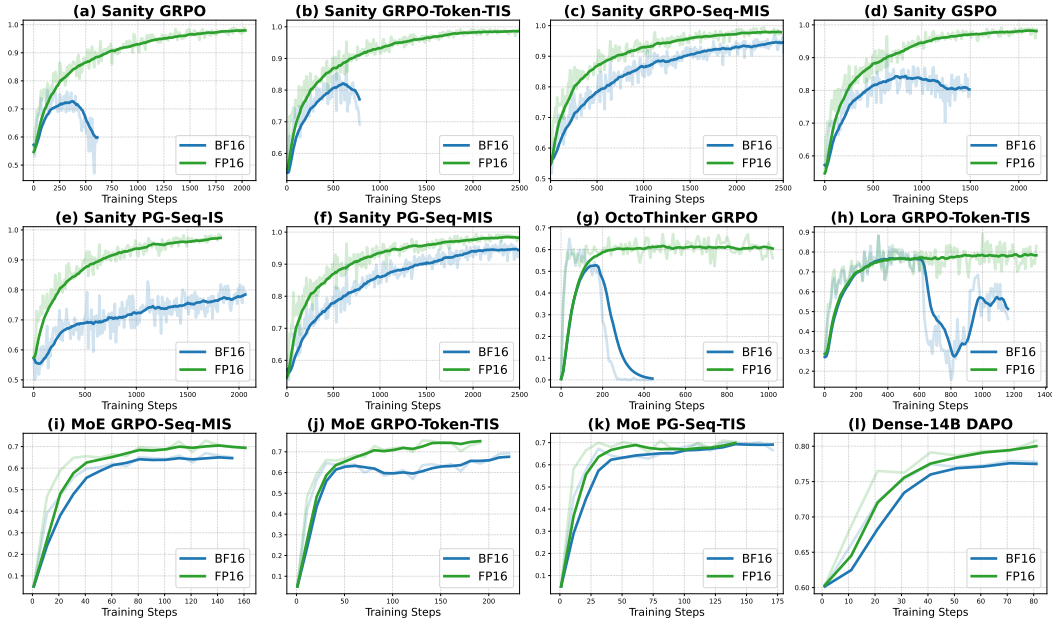


Figure 1: Training reward comparison between BF16 and FP16. We evaluate across diverse settings: our Sanity test (Section 4) with various algorithms (GRPO, GSPO, TIS, MIS, PG); different model families (RID, Qwen and OctoThinker); alternative fine-tuning methods (Lora); and larger scale models (Dense-14B, MoE). Results are validated on two independent frameworks (VeRL and Oat).

*Core Contributors.

†Project Lead.

1 Introduction

Reinforcement learning (RL) has emerged as a powerful paradigm for fine-tuning large language models (LLMs) to boost the reasoning performance [Guo et al., 2025, Zeng et al., 2025, Liu et al., 2025c, Qi et al., 2025]. However, the path to achieving high-performing models through RL is often fraught with instability. The training process is notoriously sensitive to hyperparameters and can suffer from training collapse, making it a significant challenge to reliably improve model performance [Yao et al., 2025, Liu et al., 2025a, Team et al., 2025a, Zheng et al., 2025, Yu et al., 2025, Cui et al., 2025]. This fragility has spurred a continuous search for methods that can stabilize and streamline the RL fine-tuning process.

A critical source of this instability stems from a fundamental discrepancy in modern RL frameworks: the **training-inference mismatch**. To accelerate training, these frameworks typically use different computational engines, a highly optimized one for fast inference (rollout) and another for training (gradient computation). While mathematically identical, these engines produce numerically different outputs due to precision errors and hardware-specific optimizations. As recent work has highlighted [Yao et al., 2025, Liu et al., 2025a, Team et al., 2025a], this seemingly minor mismatch between the inference and the training introduces significant issues into the optimization process.

Existing solutions have attempted to address this mismatch through algorithmic patches based on importance sampling. Notably, Yao et al. [2025] introduced a token-level importance sampling ratio as a patch to the GRPO [Shao et al., 2024] gradient. While this simple correction can prolong training, it was later shown by Liu et al. [2025a] to be insufficient to fully stabilize training due to its biased gradient. As an alternative, they proposed using an unbiased, sequence-level importance sampling ratio for the correction. Although this method is more stable, its effectiveness is hampered by slow convergence speed, a direct consequence of the high variance inherent in sequence-level ratios. Furthermore, both of these algorithmic approaches suffer from two fundamental problems:

1. **They are computationally inefficient.** The implementations from Yao et al. [2025] and Liu et al. [2025a] require an extra forward pass to compute the importance sampling ratio for their correction. Assuming a backward pass is twice the cost of a forward pass [Qi et al., 2023], this adds approximately 25% to the training cost.
2. **The deployment gap persists.** By design, these solutions correct for the mismatch during training, but the final model parameters are optimized with respect to the training engine’s probability distribution. This means the resulting model is not truly optimal for the inference engine used in deployment, which can lead to a tangible performance drop. This calls for a solution that eliminates the mismatch at its source, rather than merely compensating for it.

In this work, we take a step back from the complex algorithmic fixes and investigate the root cause of the numerical mismatch: **floating-point precision**. We identify that the modern standard for mixed-precision training, BFloat16 (BF16), is the primary culprit. While BF16 has a wide dynamic range which is excellent for stable pre-training, its low precision makes it highly susceptible to rounding errors that accumulate and eventually cause the training and inference policies to diverge.

Our key finding is super simple: by switching from **BF16 to the FP16** during RL fine-tuning, we can virtually **eliminate the training-inference mismatch**. With more mantissa bits, FP16 offers higher numerical precision, making results less sensitive to the implementation differences between training and inference. The benefits of this simple change are multifold. It eliminates the complex algorithmic workarounds and the accompanying probability evaluations, restoring RL to its purest importance weighted policy-gradient form. It also closes the deployment gap that none of the existing fixes address. Empirical evaluations show a significant and uniform boost over both performance and stability, presenting a clean, efficient, and universally applicable solution to a critical challenge in RL-based LLM alignment.

2 Background

In modern RL frameworks for LLM fine-tuning, different engines are used for inference and training to maximize system efficiency, which inevitably creates a mismatch between the inference policy $\mu(\cdot|\theta)$ and training policy $\pi(\cdot|\theta)$ due to subtle numerical discrepancies, even though, in principle, the two should be mathematically identical ($\mu = \pi$). This mismatch brings two issues elaborated below,

Biased Gradient To optimize the trainer policy $\pi(\cdot|\theta)$, we typically adopt the following objective:

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim p_{\mathcal{X}}} [\mathcal{J}(x, \theta)] = \mathbb{E}_{x \sim p_{\mathcal{X}}} [\mathbb{E}_{y \sim \pi(\cdot|x, \theta)} [R(x, y)]] , \quad (1)$$

where x is the prompt sampled from a distribution $p_{\mathcal{X}}$, y is the response, and $R(x, y)$ is the reward of y . The policy gradient can be calculated by REINFORCE estimator [Williams, 1992, Sutton and Barto, 2018]:

$$\begin{aligned} \nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{x \sim p_{\mathcal{X}}} [\nabla_{\theta} \mathcal{J}(x, \theta)] , \\ \nabla_{\theta} \mathcal{J}(x, \theta) &= \mathbb{E}_{y \sim \pi(\cdot|x, \theta)} [\nabla_{\theta} \log \pi(y|x, \theta) \cdot R(x, y)] . \end{aligned} \quad (2)$$

In practice, we sample the responses from the inference policy μ , instead of the training policy π . As noted by Yao et al. [2025] and Liu et al. [2025a], the policy gradient would become biased if simply ignoring this mismatch.

$$\begin{aligned} \nabla_{\theta} \mathcal{J}_{\text{biased}}(x, \theta) &= \mathbb{E}_{y \sim \mu(\cdot|x, \theta)} [\nabla_{\theta} \log \pi(y|x, \theta) \cdot R(x, y)] \\ &\neq \nabla_{\theta} \mathcal{J}(x, \theta) \end{aligned} \quad (3)$$

Deployment Gap Another important but hard to fix issue is the deployment gap. Though it is $\pi(\cdot|\theta)$ that we train, it is $\mu(\cdot|\theta)$ that we use for deployment and evaluation. However, the parameter θ optimized under the training engine π is not necessarily optimal for the inference engine μ :

$$\arg \max_{\theta} \mathbb{E}_{x \sim p_{\mathcal{X}}, y \sim \mu(\cdot|x, \theta)} [R(x, y)] \neq \arg \max_{\theta} \mathbb{E}_{x \sim p_{\mathcal{X}}, y \sim \pi(\cdot|x, \theta)} [R(x, y)] \quad (4)$$

This deployment gap results in a non-trivial performance degrade due to this mismatch. While algorithmic patches [Yao et al., 2025, Liu et al., 2025a] fix the biased gradient, by nature they cannot close the deployment gap, which calls for a fundamental solution to remove the mismatch altogether.

2.1 Correcting Biased Gradient via Importance Sampling

To correct the biased gradient introduced by the training-inference mismatch, a principled approach is to use importance sampling (IS). This method re-weights the gradient calculation using a sequence-level probability ratio, ensuring the gradient estimator remains unbiased. The policy gradient for a given prompt x is thus corrected as:

$$\nabla_{\theta} \mathcal{J}_{\text{pg-is}}(x) = \mathbb{E}_{y \sim \mu(\cdot|x, \theta')} \left[\frac{\pi(y|x, \theta)}{\mu(y|x, \theta')} \nabla_{\theta} \log \pi(y|x, \theta) \cdot A(x, y) \right] , \quad (5)$$

where θ' denotes the parameters used for sampling, which may differ from θ in an off-policy setting. The term $A(x, y) = R(x, y) - B(x)$ is the advantage, with $B(x)$ serving as a baseline for variance reduction [Sutton and Barto, 2018].

While theoretically sound, this estimator often suffers from high variance, particularly in the context of LLMs where response sequences are long, leading to extreme probability ratios. To mitigate this, techniques that trade a small amount of bias for a significant reduction in variance, such as Truncated Importance Sampling (TIS) [Espeholt et al., 2018, Yao et al., 2025] and Masked Importance Sampling (MIS) [Zheng et al., 2025, Team et al., 2025b, Liu et al., 2025a], have been proposed:

$$\nabla_{\theta} \mathcal{J}_{\text{pg-tis}}(x) = \mathbb{E}_{y \sim \mu(\cdot|x, \theta')} \left[\min \left(\frac{\pi(y|x, \theta)}{\mu(y|x, \theta')}, C \right) \cdot \nabla_{\theta} \log \pi(y|x, \theta) \cdot A(x, y) \right] , \quad (6)$$

$$\nabla_{\theta} \mathcal{J}_{\text{pg-mis}}(x) = \mathbb{E}_{y \sim \mu(\cdot|x, \theta')} \left[\frac{\pi(y|x, \theta)}{\mu(y|x, \theta')} \cdot \mathbb{I} \left\{ \frac{\pi(y|x, \theta)}{\mu(y|x, \theta')} \leq C \right\} \cdot \nabla_{\theta} \log \pi(y|x, \theta) \cdot A(x, y) \right] , \quad (7)$$

where C is a clipping hyperparameter and $\mathbb{I}\{\cdot\}$ is the indicator function. These methods stabilize training by controlling the magnitude of the importance weights.

2.1.1 Existing Implementations

Although generally inspired by the importance sampling principle, recent methods [Yao et al., 2025, Liu et al., 2025a] are effectively implemented as auxiliary patches on top of GRPO, rather than

adhering to the strictly principled formulation. Unfortunately, many widely used RL frameworks (e.g., VeRL [Sheng et al., 2024]) are GRPO-centric and do not natively provide the standard importance-weighted estimators outlined in Equation (5), Equation (6), and Equation (7).

The standard GRPO gradient [Shao et al., 2024, Liu et al., 2025c], which does not correct for the training-inference mismatch, is calculated as follows:¹

$$\nabla_{\theta} \mathcal{J}_{\text{grpo}}(x) = \mathbb{E}_{y \sim \mu(\cdot|x, \theta')} \left[\sum_{t=1}^{|y|} \nabla_{\theta} \min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad (8)$$

where $r_t = \frac{\pi(y_t|x, y_{<t}, \theta)}{\pi(y_t|x, y_{<t}, \theta')}$ and $A_t = R(x, y) - \frac{1}{G-1} \sum_{i=1}^{G-1} R(x, y_i)$.

For each prompt x , a group of G responses $\{y_i\}_{i=1}^G$ is sampled from the inference policy $\mu(\cdot|x, \theta')$ to compute the advantage function A_t as in GRPO and RLOO [Ahmadian et al., 2024, Kool et al., 2019].

Based on GRPO, Yao et al. [2025] introduced a token-level TIS correction:

$$\nabla_{\theta} \mathcal{J}_{\text{grpo-tok-tis}}(x) = \mathbb{E}_{y \sim \mu(\cdot|x, \theta')} \left[\sum_{t=1}^{|y|} \min(\rho_t, C) \cdot \nabla_{\theta} \min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad (9)$$

where $\rho_t = \frac{\pi(y_t|x, y_{<t}, \theta')}{\mu(y_t|x, y_{<t}, \theta')}$.

Subsequently, Liu et al. [2025a] advanced this approach by proposing a sequence-level MIS variant. This correction is applied to the entire GRPO gradient term, using a single ratio for the whole sequence to determine whether the update is applied:

$$\nabla_{\theta} \mathcal{J}_{\text{grpo-seq-mis}}(x) = \mathbb{E}_{y \sim \mu(\cdot|x, \theta')} \left[\rho \cdot \mathbb{I}\{\rho \leq C\} \cdot \sum_{t=1}^{|y|} \nabla_{\theta} \min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad (10)$$

where $\rho = \frac{\pi(y|x, \theta')}{\mu(y|x, \theta')}$.

Compared to the vanilla policy gradient estimators (Equation (5) and its TIS/MIS variants), existing GRPO-based implementations require an additional forward pass to compute $\pi(\cdot|\theta')$ for their off-policy correction. This extra step incurs approximately 25% computational overhead during training, assuming a backward pass is twice as costly as a forward pass [Qi et al., 2023].

2.2 Engineering Attempts to Reduce the Mismatch

Another line of work attempts to mitigate the training-inference mismatch from an engineering perspective, but with limited success. Early attempts, such as using an FP32 language model head by Chen et al. [2025], is shown to be insufficient to prevent training collapse [Yao et al., 2025, Liu et al., 2025a]. Very recently, Team et al. [2025a] reported promising results by manually aligning training and inference implementations. However, this approach requires deep domain knowledge and substantial engineering effort, and it is unclear whether such bespoke fixes can be generalized across different frameworks or models. A tangentially related work by He [2025] demonstrated how to enforce determinism in inference, their method incurs a significant efficiency cost and cannot directly address the training-inference mismatch.

Despite these engineering efforts, the mismatch persists due to fundamental differences between training and inference computations that are difficult to reconcile. For example, tokens are generated auto-regressively during inference but are processed in parallel during training. Different parallelization strategies and precision-sensitive operations such as top-k expert selection in Mixture-of-Experts (MoE) models, further complicate the situation. This inherent difficulty highlights the need for a more fundamental solution that avoids such complex and brittle engineering workarounds.

¹We use the Dr.GRPO variant to remove the length and difficulty biases of the vanilla GRPO.

3 Revisiting FP16 Precision

In our investigation of the training–inference mismatch, we identify a surprisingly simple yet highly effective remedy that avoids complex algorithmic or engineering fixes. Rather than introducing additional machinery, we focus on a more fundamental factor: numerical precision. We find that merely switching the training precision from the now-dominant BF16 format [Dean et al., 2012, Kalamkar et al., 2019] to the earlier Float16 (FP16) format [Micikevicius et al., 2017] substantially mitigates the policy mismatch and yields significant performance improvements across RL algorithms. This section revisits the history and characteristics of these floating-point formats to shed light on this counterintuitive but powerful result.

3.1 FP16 vs. BF16

Floating-point formats represent real numbers by dividing their bit budget between two components: exponent bits, which determine the range (how large or small a value can be), and mantissa bits (also known as fraction bits), which determine the precision (how finely values can be distinguished within that range). Both FP16 and BF16 use 16 bits in total, but they allocate these bits differently, resulting in distinct trade-offs between range and precision (see Table 1).

FP16 (IEEE 754 half-precision) allocates 5 bits to the exponent and 10 bits to the mantissa. The relatively large mantissa gives FP16 higher numerical precision, allowing it to represent small differences between nearby values accurately. However, its limited 5-bit exponent severely constrains the dynamic range, making FP16 prone to overflow (values exceeding the representable maximum) and underflow (values rounding to zero). Training with FP16 often requires stability techniques such as loss scaling to mitigate these issues (see Section 3.2).

BF16 (bfloat16), introduced by Google, allocates 8 bits to the exponent—matching the range of the 32-bit FP32 format—and only 7 bits to the mantissa. This design provides a wide dynamic range comparable to FP32, making BF16 highly resistant to overflow and underflow, at the cost of reduced precision. The resulting numerical robustness under low precision is the key reason for its widespread adoption in large-scale deep learning systems.

Table 1: Comparison of 16-bit Floating-Point Formats.

Property	FP16	BF16
<i>Bit Allocation</i>		
Exponent Bits	5	8
Mantissa Bits	10	7
<i>Dynamic Range</i>		
Smallest Positive Normal	$\approx 6.1 \times 10^{-5}$	$\approx 1.2 \times 10^{-38}$
Largest Value	$\approx 6.6 \times 10^4$	$\approx 3.4 \times 10^{38}$
<i>Precision</i>		
Next Representable > 1	$1 + 2^{-10} \approx 1.000977$	$1 + 2^{-7} \approx 1.007812$

3.2 Stabilizing FP16 Training with Loss Scaling

The primary challenge with FP16’s limited range is gradient underflow, which can be effectively solved early in the history of mixed-precision training with a technique called **loss scaling** [Micikevicius et al., 2017]. The procedure is straightforward:

1. The loss is multiplied by a large scaling factor S before backpropagation.
2. This scales up all gradients by S , shifting small gradient values out of the underflow region and into the representable range of FP16, thus preserving them.
3. Before updating the weights, the gradients are scaled back by dividing S .

Modern implementations have further improved this with dynamic loss scaling. The scaling factor S is automatically adjusted during training, increased if no overflows (infinity values in gradients) are detected for a number of steps, and decreased immediately if an overflow occurs.

Table 2: Evaluation scores of DeepSeek-R1-Distill-Qwen-1.5B using under different precisions (BF16, FP16 and FP32) and token budgets (8K and 32K).

dtype	AMC23 (8K)	AIME24 (8K)	AMC23 (32K)	AIME24 (32K)
BF16	50.38	22.60	62.35	29.90
FP16	50.60	20.10	63.10	30.94
FP32	51.54	22.30	62.42	28.44

Crucially, these loss scaling techniques are standard, mature components in mainstream training frameworks (e.g., PyTorch [Paszke et al., 2019], Megatron [Shoeybi et al., 2019], DeepSpeed [Rasley et al., 2020]). Enabling them typically requires **only a single configuration change or a few lines of code**, making the adoption of FP16 training both simple and robust.

3.3 The Rise of BF16 in Modern LLM Training

Despite the effectiveness of loss scaling, it complicates the system in distributed settings. Because a global synchronization is needed before the optimizer step to check for overflows and ensure the scaling factor is aligned across all workers.

The introduction of BF16 on hardware like Google TPUs and later NVIDIA GPUs (starting with the Ampere architecture) is a game-changer. Having a same dynamic range as FP32, BF16 offered a “drop-in” replacement for FP32 that obviates meticulous loss scaling. Its resilience to overflow and underflow made training LLMs significantly more stable and straightforward. Consequently, BF16 quickly became the de-facto standard for modern mixed-precision training.

3.4 Why FP16 is the Key for RL Fine-Tuning

While BF16’s stability is an advantage for pre-training models, our findings reveal that its low precision is the origin of the training-inference mismatch.

Modern RL frameworks often use different engines or optimized kernels for training and inference. Even if both are configured to use BF16, subtle differences in their implementation (e.g., CUDA kernel optimizations, parallel strategies) can lead to different rounding errors on BF16. When these small discrepancies accumulate over a sequence of tokens during autoregressive sampling, the resulting probability distributions for π and μ can diverge significantly. This divergence is the source of the biased gradients and the deployment gap discussed earlier.

This is precisely why switching to FP16 provides a fundamental solution. With its 10 mantissa bits, FP16 offers 8 times more precision (2^{10} values vs. 2^7 values) than BF16. This higher fidelity means that the outputs of the training and inference engines are much more likely to be numerically identical. The increased precision creates a buffer that absorbs the minor implementation differences between the two engines, preventing rounding errors from accumulating and causing a policy divergence.

For RL fine-tuning, the dynamic range of the model’s weights and activations has already been established during pre-training. Therefore, the extreme range of BF16 is less critical, while the precision it sacrifices becomes a dominant drawback. By reverting to FP16, we trade the unnecessary range of BF16 for the critical precision, effectively closing the gap between training and inference without any complex algorithmic or engineering workaround.

3.5 Offline Analysis Results

Before proceeding to RL fine-tuning, we first perform an offline analysis to examine performance and training–inference mismatch under different numeric precisions. We begin by sampling 32 responses per question from the AMC and AIME benchmarks [Li et al., 2024] using the DeepSeek-R1-Distill-Qwen-1.5B model² [Guo et al., 2025], with a 32K total token budget under both BF16 and FP16 precisions. As shown in Table 2, their performance is largely comparable, suggesting that higher inference precision alone does not necessarily yield improvements.

²We follow their recommended decoding settings: temperature 0.6 and top- p 0.95.

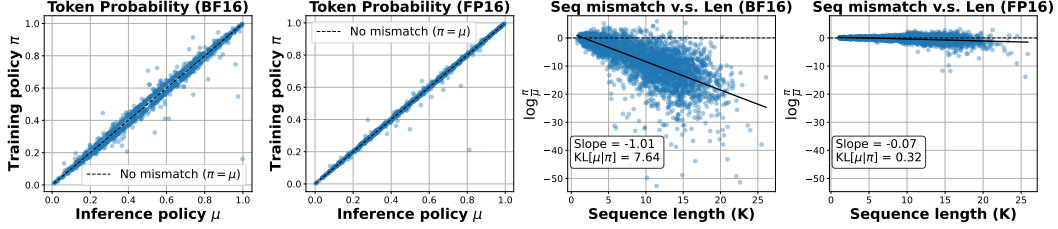


Figure 2: FP16 significantly reduces the training-inference mismatch. The left two plots show the token-level probability distribution, and the right two plots present the distribution of sequence-level log probability ratio between the inference policy (μ) and the training policy (π). Dashed lines in black denote perfect precision without mismatch.

Next, we re-generate 32 responses per question using temperature 1.0 and no top- p sampling (so that μ is directly comparable to π), and evaluate the token log-probabilities using the same model weights within the DeepSpeed training engine, under both BF16 and FP16 settings. The left two plots in Figure 2 show the resulting distributions of token probabilities. We find that FP16 notably reduces the mismatch between μ and π , with data points more tightly concentrated around the diagonal.

Beyond token-level discrepancies, we also analyze sequence-level mismatch, since $\frac{\pi(y|x)}{\mu(y|x)}$ serves as an unbiased estimator of the importance sampling weight for a full response. The right two plots in Figure 2 depict the distribution of sequence-level log-probability ratios across different generation lengths. The results clearly indicate that BF16 introduces an **exponentially larger mismatch**, which worsens with longer responses due to cumulative autoregressive errors, whereas FP16 maintains the mismatch at a much milder level (approximately $24\times$ smaller).

4 A Sanity Test for RL Algorithms

To rigorously assess the reliability and robustness of RL algorithms, we introduce a novel sanity test. Standard benchmarks often contain a mix of problems with varying difficulty, including questions that are either overly trivial or unsolvable by the initial model. Trivial questions waste computational resources, while unsolvable ones make it difficult to determine whether poor performance stems from a flawed algorithm or the model’s inherent limitations. **Our sanity test is designed to remove this ambiguity with efficiency. By creating a perfectible dataset where every problem is known to be solvable but not trivial, we can cleanly isolate and evaluate an RL algorithm’s ability to unlock a model’s latent potential.** On this perfectible dataset, **a reliable RL algorithm should theoretically be able to achieve 100% training accuracy.**

We construct this perfectible dataset by filtering out those overly trivial and unsolvable questions for the initial model. Specifically, we unroll 40 responses for each problem in the MATH dataset [Hendrycks et al., 2021], and only keep problems where the initial accuracy is between 20% and 80%. This process yielded a targeted dataset of 1,460 questions for the DeepSeek-R1-Distill-Qwen-1.5B model [Guo et al., 2025]. The smaller size of this dataset makes achieving near-100% accuracy computationally feasible, allowing for efficient and conclusive testing.

We define our sanity test with a clear criterion: an RL algorithm passes if its training accuracy on this perfectible dataset converges above a high threshold (e.g., 95%). An algorithm that fails this test can be considered unreliable or fundamentally flawed, as it is unable to guide the model to solve problems known to be within its reach. While passing is not a guarantee of universal success, failing is a strong indicator of an ill-suited algorithm design, making this test a crucial diagnostic tool.

4.1 Experimental Setup

Under this sanity test, we evaluate several representative RL algorithms, particularly those designed to address the training-inference mismatch (see Section 2.1). **All experiments use DeepSeek-R1-Distill-Qwen-1.5B as the initial model, with a context length of 8,000.** We run each experiment on 8 NVIDIA A100 80G GPUs. For each policy iteration [Schulman et al., 2017], we use a batch size of 64 questions (with 8 rollouts per question) and perform 4 gradient steps. For algorithms in the

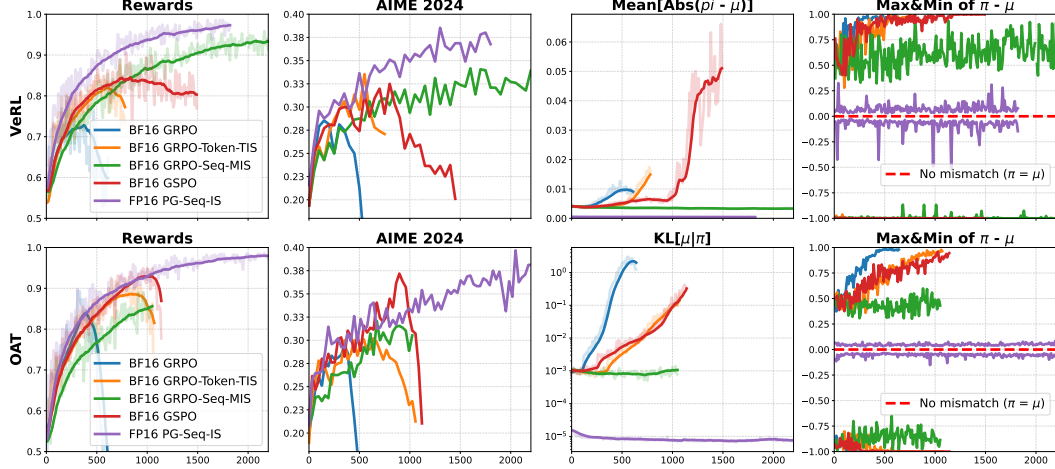


Figure 3: Simply switching from BF16 to FP16 stabilizes and prolongs RL training. The basic importance-weighted policy gradient algorithm in FP16 outperforms all baselines in BF16. Note that the third metric reported in each row slightly differs in implementation due to the use of separate codebases (VeRL and Oat). These metrics are semantically similar, and the minor differences do not affect our conclusions.

GRPO family, we set the `clip_higher` to 0.28 by default [Yu et al., 2025]. The clipping threshold for importance sampling methods (Equation (7) and Equation (10)) is set to $C = 3$.

We evaluate a suite of methods designed to address the training-inference mismatch. This includes:

- A vanilla GRPO baseline (specifically, the Dr.GRPO variant from Equation (8)) [Shao et al., 2024, Liu et al., 2025c].
- GRPO with a token-level TIS correction (Equation (9)) from Yao et al. [2025].
- GRPO with a sequence-level MIS correction (Equation (10)) from Liu et al. [2025a].
- The standard policy gradient algorithm with importance sampling (Equation (5)).

In addition, we include GSPO [Zheng et al., 2025] in our experiments, although it was primarily designed to address the mismatch introduced by MoE models.

4.2 Comparison with Existing Algorithmic Corrections

To ensure robustness and rule out implementation-specific artifacts, we conducted experiments across two different frameworks: VeRL³ [Sheng et al., 2024] and Oat [Liu et al., 2025b]. The results, shown in Figure 3, highlight the instability of existing methods when using BF16 precision.

The vanilla GRPO baseline collapses early in training, reaching a peak accuracy of only 73% in VeRL and 84% in Oat before its performance degrades. The token-level TIS correction [Yao et al., 2025] prolongs training slightly but ultimately fails, collapsing after reaching 82% (VeRL) and 88% (Oat) accuracy, an observation that aligns with findings from Liu et al. [2025a]. Surprisingly, GSPO demonstrates more stable training for a longer period than GRPO with token-level TIS, achieving higher rewards despite not using the inference policy μ at all.⁴

Among all the algorithmic corrections in BF16, only GRPO with sequence-level MIS [Liu et al., 2025a] maintains stable training without collapsing. However, this stability is costly. The method suffers from slow convergence due to the high variance of its sequence-level importance ratio (see Figure 2). More importantly, even at its peak, it exhibits a **significant deployment gap** compared to

³We identified and corrected an implementation bug in VeRL’s Dr.GRPO for our experiments. We optimized the training speed of VeRL based on <https://github.com/sail-sg/odc>.

⁴In our VeRL experiment, the GSPO gradient norm became ‘NaN’ after 1200 steps, halting further model updates.

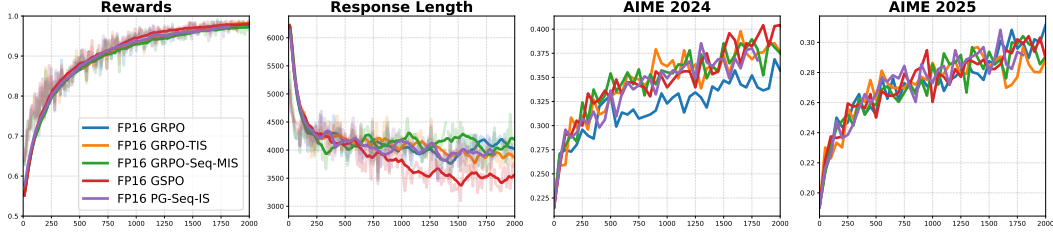


Figure 4: Comparisons between various algorithms based on FP16.

our FP16 approach. It achieves a maximum training accuracy of only 95% (vs. 99% in FP16) and a score of 34% (vs. 39% in FP16) on the AIME 2024 benchmark, demonstrating a clear performance ceiling. More evidence on deployment gap can be found in Figures 1 and 6.

The Efficacy of FP16 Precision In contrast to these algorithmic approaches, simply switching both training and inference precision from BF16 to FP16 provides a dramatic improvement. As shown in Figures 1 and 6, the FP16 training runs are significantly more stable, converge much faster, and achieve substantially higher final rewards and evaluation scores across all tested algorithms. This result demonstrates that addressing the mismatch at the precision level is a more direct and effective solution than applying unstable or inefficient algorithmic corrections.

The most surprising finding is that FP16 precision fundamentally improves the behavior of importance sampling. The sequence-level ratio, which is notoriously high-variance, becomes much more concentrated and stable in FP16 (see Figure 2). This stabilization **makes it practical to use the classic, unbiased policy gradient estimator without any modifications** (Equation (5)). As shown in Figure 3, this simple, unbiased approach, when powered by FP16, dramatically outperforms all existing algorithmic corrections in BF16.

Training Dynamics Our experimental results reveal an interesting phenomenon: algorithms that eventually collapse consistently exhibit a growing training-inference mismatch beforehand, making it a potential early-warning signal (see Figure 3). During this period, the policy difference $\pi(\cdot|\theta') - \mu(\cdot|\theta')$ also converges to extreme values, where one policy’s probability approaches 1 while the other’s approaches 0, despite using the same copy of weights. We suspect this is driven by a particular optimization bias, though further validation is required. In contrast, stable algorithms maintain a bounded mismatch. Crucially, FP16 training shows a much lower mismatch level than any BF16 method. This inherent stability at the precision level explains why a simple policy gradient with FP16 can outperform all existing, more sophisticated solutions.

Framework-Specific Differences While our core conclusions hold across both the VeRL [Sheng et al., 2024] and Oat [Liu et al., 2025b] frameworks, we observed subtle implementation-dependent differences. Initially, the training-inference mismatch is slightly smaller in Oat than in VeRL; for example, the initial policy difference $\pi(\cdot|\theta') - \mu(\cdot|\theta')$ has a minimum near -0.9 in Oat versus -1.0 in VeRL. Even under FP16, where both frameworks exhibit a small mismatch, VeRL was more prone to occasional numerical spikes. These subtle stability differences, which we attribute to their different distributed backends (DeepSpeed ZeRO vs. PyTorch FSDP), likely explain why Oat yields slightly higher training rewards, particularly for the algorithms that eventually collapse.

4.3 Reviewing RL Algorithms under FP16

We then reviewed the performance of various RL algorithms when trained with FP16 precision. As shown in Figure 4, the performance differences between algorithms become almost indistinguishable. We attribute this convergence in performance to the significantly reduced training-inference mismatch in FP16, which effectively transforms the optimization problem into a nearly on-policy setting. In this state, the complex corrections offered by different algorithms provide little to no additional benefit. We did observe a minor exception where the original GRPO scored slightly lower on the AIME 2024 benchmark; however, it also scored slightly higher on AIME 2025, making it difficult to draw a definitive conclusion about its relative performance.

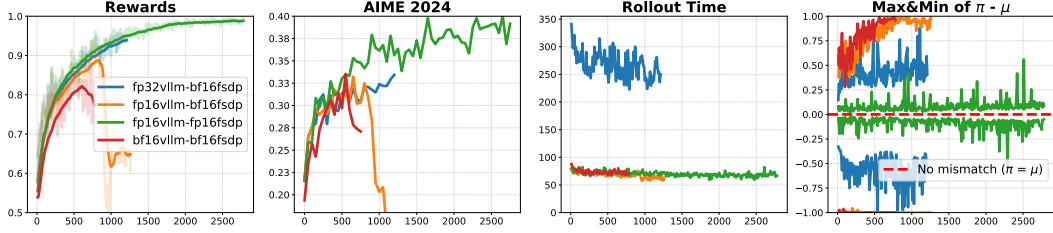


Figure 5: Ablation on the precision combinations.

4.4 Ablation on the Precision

To isolate the effects of training and inference precision, we conducted an ablation study on the VeRL framework, using vLLM [Kwon et al., 2023] for inference and PyTorch FSDP [Zhao et al., 2023] for training. The results are presented in Figure 5.

When training with BF16 precision, we found that increasing the inference precision consistently prolonged training stability and improved performance. Notably, when paired with FP32 inference, the training run became fully stable with no signs of collapse. However, this stability came at an immense cost: FP32 inference was nearly three times slower than FP16 or BF16 inference, making this combination impractical for large-scale experiments.

In contrast, using FP16 for both training and inference yielded the best results. This combination not only produced the lowest training-inference mismatch but also resulted in the most stable training dynamics. It successfully reached nearly 100% training accuracy on the perfectible dataset without any loss of speed, demonstrating a clear superiority in both stability and efficiency.

5 Generalization Across Models, Data, and Training Regimes

In Section 4, we scrutinized various algorithmic fixes under the sanity-check setting and found that simply switching from BF16 to FP16 can substantially improve training stability (Section 4.2), with its effect often overshadowing algorithmic tweaks (Section 4.3). In this section, we move beyond the sanity-check setting and validate our findings across more diverse scenarios, including Mixture-of-Experts (MoE) RL, Low-Rank Adaptation (LoRA) RL, and RL on larger prompt sets and alternative model families.

5.1 MoE RL

Mixture-of-Experts (MoE) reinforcement learning (RL) training is known for its instability and often requires sophisticated stabilization strategies [Zheng et al., 2025]. Both training and inference of MoE models typically involve distinct parallelization strategies and precision-sensitive operations such as top- k expert selection, which further complicate the situation and usually lead to a larger training-inference mismatch compared to dense models. Given the widespread adoption of MoE architectures in modern LLMs, we conduct RL experiments on MoE models using Qwen3-30B-A3B-Base. We evaluate three different algorithms: GRPO-Seq-MIS, GRPO-Token-TIS, and PG-Seq-TIS, with detailed experimental settings provided in Section A.1.

Experiments using FP16 show greater stability and consistently higher training accuracies (see (i), (j), and (k) in Figure 1) as well as higher validation rewards (see (i), (j), and (k) in Figure 6). The improvement is consistent across all three algorithms, indicating that adopting FP16 effectively mitigates the training-inference mismatch and enhances overall performance.

5.2 LoRA RL

LoRA [Hu et al., 2022] has recently regained popularity in LLM RL [Wang et al., 2025a, Schulman and Lab, 2025] due to its efficiency and performance comparable to full fine-tuning. To examine how LoRA-based RL is affected by numeric precision, we train Qwen2.5-Math-1.5B models on the standard MATH dataset using GRPO-Token-TIS (Equation (9)). LoRA is applied to all layers with a

rank of 32 and scaling factor $\alpha = 64$. Following Schulman and Lab [2025], we adopt a slightly larger learning rate (4×10^{-5}) than that used in full fine-tuning. As shown in Figure 1 (h), BF16-based LoRA training collapses after roughly 600 steps, whereas FP16 maintains stable training throughout.

5.3 RL on Large Dense Models

Large-scale parameters are typically required in modern LLMs, yielding significantly better performance compared to smaller models. This motivates us to conduct RL experiments on large dense models. Specifically, we experiment with Qwen3-14B-Base and follow the algorithm of DAPO [Yu et al., 2025]. Refer to Section A.1 for details of experimental settings.

As shown in Figure 1 (l), the training rewards with FP16 increase much faster than those with BF16. Figure 6 (l) demonstrates that FP16 achieves higher validation accuracy on AIME 2024. These results suggest that using FP16 instead of BF16 effectively mitigates the training–inference mismatch in large models, highlighting the potential of this approach for scaling RL training on large models.

5.4 RL on Other Model Families

The base models, which serve as the initial policies for RL, can substantially influence the learning dynamics, as they determine not only the scope of exploration but also the numerical range and sensitivity of network parameters and activations. To strengthen our experimental conclusions, we extend our study beyond Qwen-based models and train OctoThinker-3B [Wang et al., 2025b], a model mid-trained from Llama3.2-3B [Grattafiori et al., 2024] on reasoning-intensive data using GRPO. As shown in Figure 1 (g), BF16 training destabilizes after around 150 steps due to numerical mismatch, while FP16 continues to train smoothly without collapse.

6 Discussions

Rethinking the Precision Tradeoff in RL Fine-Tuning Numerical precision is a foundational choice in the LLM training stack, yet this choice has long been dominated by BF16 for both pre-training and post-training, prized for its wide dynamic range and ease of use. Our results, however, suggest this default deserves careful rethinking for RL fine-tuning. In this phase, the training–inference mismatch becomes a critical source of instability, and BF16’s low precision exacerbates this problem. We demonstrate that by simply trading BF16’s wide dynamic range for FP16’s higher precision, one can achieve significantly more stable RL training, faster convergence, and superior final performance.

It is important to note that we are not claiming FP16 is a universally optimal choice. The pursuit of efficiency may lead developer to even lower precisions like FP8. Furthermore, using FP16 for extremely large models might present engineering challenges related to its limited range, such as managing potential overflows. However, we believe these are solvable challenges, as evidenced by the recent successes in large-scale FP8 training. Ultimately, we hope this work inspires the community to reconsider FP16 as a powerful and often more suitable alternative for stabilizing RL fine-tuning.

The Bias-Variance Tradeoff under BF16 Precision Our results in Section 4.2 reveal a bias–variance trade-off among RL algorithms operating under BF16 precision. Methods with lower variance but higher bias (like GRPO, token-level TIS, and GSPO) initially converge quickly but prove unstable and eventually collapse. Conversely, less biased algorithms that more accurately correct for the policy mismatch (like PG-Seq-IS and GRPO-Seq-MIS) achieve stability but at the cost of high variance, which slows their convergence.

This trade-off, however, becomes far less critical under FP16 precision. By fundamentally reducing the training–inference mismatch, FP16 naturally lowers both the bias induced by the mismatch and the variance of the importance sampling corrections. This enhanced stability allows even the most naive policy gradient estimator to converge efficiently, creating a training dynamic where all tested algorithms perform well and the tension between stability and speed is effectively resolved.

7 Conclusion

This work demonstrates that the training-inference mismatch, a major source of instability in RL fine-tuning, is fundamentally a problem of numerical precision. While existing algorithmic fixes are often complex and inefficient, we show that simply switching from the standard BF16 format to the higher-precision FP16 format can virtually eliminate the mismatch. This single, efficient change leads to more stable training, faster convergence, and superior performance, proving that addressing the problem at the precision level is a more effective strategy. We conclude that FP16 should be reconsidered as a foundational option for robust RL fine-tuning of LLM.

References

- Arash Ahmadian, Chris Cremer, Matthias Gall , Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet  st n, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.
- Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.
- Zhoujun Cheng, Shibo Hao, Tianyang Liu, Fan Zhou, Yutao Xie, Feng Yao, Yuexin Bian, Yonghao Zhuang, Nilabjo Dey, Yuheng Zha, Yi Gu, Kun Zhou, Yuqi Wang, Yuan Li, Richard Fan, Jian-shu She, Chengqian Gao, Abulhair Saparov, Haonan Li, Taylor W. Killian, Mikhail Yurochkin, Zhengzhong Liu, Eric P. Xing, and Zhiting Hu. Revisiting reinforcement learning for llm reasoning from a cross-domain perspective, 2025. URL <https://arxiv.org/abs/2506.14965>.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’ aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. Large scale distributed deep networks. *Advances in neural information processing systems*, 25, 2012.
- Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International conference on machine learning*, pages 1407–1416. PMLR, 2018.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Horace He. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- Jujie He, Jiakai Liu, Chris Yuhao Liu, Rui Yan, Chaojie Wang, Peng Cheng, Xiaoyu Zhang, Fuxiang Zhang, Jiacheng Xu, Wei Shen, Siyuan Li, Liang Zeng, Tianwen Wei, Cheng Cheng, Bo An, Yang Liu, and Yahui Zhou. Skywork open reasoner series. <https://capricious-hydrogen-41c.notion.site/Skywork-Open-Reasoner-Series-1d0bc9ae823a80459b46c149e4f51680>, 2025. Notion Blog.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

- Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. A study of bfloat16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.
- Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 reinforce samples, get a baseline for free!, 2019.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13:9, 2024.
- Jiacai Liu, Yingru Li, Yuqian Fu, Jiawei Wang, Qian Liu, and Yu Shen. When speed kills stability: Demystifying rl collapse from the inference-training mismatch, 2025a. <https://yingru.notion.site/When-Speed-Kills-Stability-Demystifying-RL-Collapse-from-the-Inference-Training-Mismatch-271211a558b7808d8b12d403fd15edda>.
- Zichen Liu, Changyu Chen, Chao Du, Wee Sun Lee, and Min Lin. Oat: A research-friendly framework for llm online alignment. <https://github.com/sail-sg/oat>, 2025b.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025c.
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://github.com/agentica-project/deepscaler>, 2025.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. Zero bubble pipeline parallelism. *arXiv preprint arXiv:2401.10241*, 2023.
- Penghui Qi, Zichen Liu, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Optimizing anytime reasoning via budget relative policy optimization. *arXiv preprint arXiv:2505.13438*, 2025.
- Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3505–3506, 2020.
- John Schulman and Thinking Machines Lab. Lora without regret. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250929. <https://thinkingmachines.ai/blog/lora/>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.
- Ling Team, Bin Han, Caizhi Tang, Chen Liang, Donghao Zhang, Fan Yuan, Feng Zhu, Jie Gao, Jingyu Hu, Longfei Li, Meng Li, Mingyang Zhang, Peijie Jiang, Peng Jiao, Qian Zhao, Qingyuan Yang, Wenbo Shen, Xinxing Yang, Yalin Zhang, Yankun Ren, Yao Zhao, Yibo Cao, Yixuan Sun, Yue Zhang, Yuchen Fang, Zibin Lin, Zixuan Cheng, and Jun Zhou. Every attention matters: An efficient hybrid architecture for long-context reasoning. *arXiv preprint arXiv:2510.19338*, 2025a.
- Ling Team, Anqi Shen, Baihui Li, Bin Hu, Bin Jing, Cai Chen, Chao Huang, Chao Zhang, Chaokun Yang, Cheng Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025b.
- Shangshang Wang, Julian Asilis, Ömer Faruk Akgül, Enes Burak Bilgin, Ollie Liu, and Willie Neiswanger. Tina: Tiny reasoning models via lora. *arXiv preprint arXiv:2504.15777*, 2025a.
- Zengzhi Wang, Fan Zhou, Xuefeng Li, and Pengfei Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling. *arXiv preprint arXiv:2506.20512*, 2025b.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training, August 2025. <https://fengyao.notion.site/off-policy-rl>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.
- Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A Detailed Experimental Settings

A.1 MoE RL

As for experiments of MoE RL, we use Qwen3-30B-A3B-Base as the base model. The training data comes from DAPO-Math-17k [Yu et al., 2025], and we conduct online evaluation on AIME 2024 using the avg@32 metric. The training is performed with the VeRL framework [Sheng et al., 2024], and the key hyperparameters are summarized in Table 3.

Dr.GRPO [Liu et al., 2025c] proposes using a constant normalizer instead of a token-count-based normalizer. Notably, the open-source VeRL implementation does not correctly implement this. We refer to our corrected version as “seq-mean-token-sum-norm” for `actor.loss_agg_mode` in VeRL.

A.2 RL on Large Dense Models

For experiments on large dense models, we use Qwen2.5-14B-Base as our base model. The training data is sourced from the mathematical domain dataset curated by Cheng et al. [2025]. They aggregated recent math reasoning collections including OR1 [He et al., 2025], DAPO [Yu et al., 2025], and DeepScaler [Luo et al., 2025], and then performed deduplication and filtering to derive a final collection of 54.4k math training samples. We conduct online evaluation on AIME 2024 using the avg@8 metric. The training algorithms and hyperparameters follow the setup described in Yu et al. [2025], as summarized in Table 3.

Table 3: Hyperparameters used for RL training of MoE models and large dense models.

Parameter	MoE RL	Large dense RL
<code>trainer.nnodes</code>	8	8
<code>trainer.n_gpu_per_node</code>	8	8
<code>model.path</code>	Qwen3-30B-A3B-Base	Qwen3-14B-Base
<code>vllm_version</code>	0.10.0	0.10.0
<code>data.train_batch_size</code>	512	512
<code>data.gen_batch_size</code>	N/A	1536
<code>data.max_prompt_length</code>	2048	2048
<code>data.max_response_length</code>	20480	20480
<code>rollout.n</code>	16	16
<code>rollout.temperature</code>	1.0	1.0
<code>rollout.top_p</code>	1.0	1.0
<code>val_kwargs.temperature</code>	0.6	1.0
<code>val_kwargs.top_p</code>	1.0	0.7
<code>actor.ppo_mini_batch_size</code>	32	32
<code>actor.ppo_max_token_len_per_gpu</code>	22528	22528
<code>optim.lr</code>	1e-6	1e-6
<code>optim.lr_warmup_steps</code>	N/A	10
<code>optim.weight_decay</code>	0.0	0.1
<code>optim.betas</code>	[0.9, 0.95]	[0.9, 0.999]
<code>optim.eps</code>	1e-15	1e-8
<code>algorithm.use_kl_in_reward</code>	False	False
<code>actor.use_kl_loss</code>	False	False
<code>actor.clip_ratio_high</code>	0.28	0.28
<code>actor.clip_ratio_low</code>	0.2	0.2
<code>actor.clip_ratio_c</code>	N/A	10.0
<i>C</i> in Equations (6) and (7)	3.0	N/A
<code>actor.loss_agg_mode</code>	seq-mean-token-sum-norm	token-mean
<code>overlong_buffer.enable</code>	False	True
<code>overlong_buffer.len</code>	N/A	4096
<code>overlong_buffer.penalty_factor</code>	N/A	1.0
<code>filter_groups.enable</code>	False	True
<code>filter_groups.metric</code>	N/A	acc
<code>filter_groups.max_num_gen_batches</code>	N/A	10

B More Experimental Results

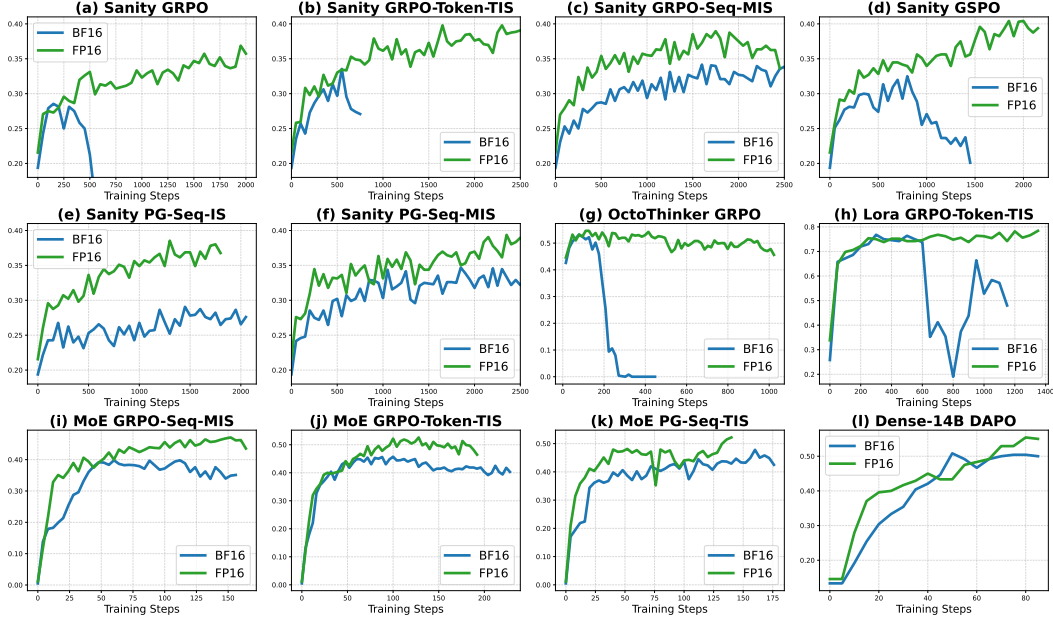


Figure 6: Evaluation comparisons between BF16 and FP16 across various frameworks, algorithms, datasets and training regimes.

While Figure 1 presents the training reward curves under different precisions, Figure 6 shows evaluation results using checkpoints trained with these precisions. The results indicate that FP16-trained models generalize well to unseen benchmarks, further supporting our claim.